



UNIVERSIDAD ESTATAL AMAZÓNICA

Unidad de Organización Curricular: Básica

Campo de Estudio: Tronco común

INFORME DE PRÁCTICAS DEL COMPONENTE PRÁCTICO-EXPERIMENTAL	
Nombre del Estudiante: Carlos Jamil Papa Tanguila	
Asignatura: Estructura de Datos	Calificación:
Fecha del informe: 24/08/2026	
N.º Práctica: 03	
Título de la Práctica: Registro y gestión de libros en una biblioteca mediante conjuntos, mapas y diccionarios	
Introducción	Las estructuras de datos permiten organizar información y seleccionar operaciones acordes con la forma en que será consultada. En un sistema de biblioteca se requiere identificar cada libro sin ambigüedad, agruparlo por autor o categoría, comprobar su disponibilidad y relacionar un préstamo con su lector. Estas necesidades corresponden a estructuras de mapeo y de conjunto, porque no basta con conservar una secuencia; también es necesario acceder mediante claves y evitar identificadores repetidos (Cormen et al., 2022). En C#, Dictionary<TKey, TValue> representa asociaciones entre claves y valores. En esta práctica el ISBN funciona como clave única del catálogo, mientras el objeto Libro conserva título, autor, categoría y año. HashSet<T> representa un conjunto sin elementos duplicados y ofrece operaciones de pertenencia de alto rendimiento; por ello se empleó para los ISBN disponibles y como valor de los mapas invertidos por autor y categoría (Microsoft, s. f.-a; Microsoft, s. f.-b). Justificación La práctica se justifica por una necesidad concreta: una biblioteca pequeña que utiliza listas manuales puede registrar dos veces el mismo ISBN, recorrer todos los libros para responder una consulta o prestar un ejemplar que ya se encuentra ocupado. La solución desarrollada reduce esos riesgos al imponer una clave única, mantener índices de consulta y separar los ISBN disponibles de los préstamos activos. Para el estudiante, el problema permite comprobar en una situación real que la elección de la estructura modifica la claridad del diseño y el costo de las operaciones. Por tanto, no se eligieron diccionarios y conjuntos solo para cumplir un requisito académico, sino porque sus propiedades responden directamente a la identificación, agrupación y control de disponibilidad de los libros. El sistema fue desarrollado en C# con programación orientada a objetos. El registro Libro modela los datos de cada obra; la clase Biblioteca encapsula el

	catálogo, los mapas, los conjuntos y las reglas; y Program administra el menú, las pruebas y la medición. El objetivo fue diseñar, implementar y comprobar una aplicación funcional que registre, consulte, preste, devuelva, reporte y guarde libros, además de analizar el tiempo de ejecución y las ventajas y limitaciones de cada estructura utilizada.
Desarrollo o procedimiento	Escenario y recursos La práctica se realizó de forma individual en un entorno doméstico de estudio, utilizando un computador con Windows, Visual Studio Code, el SDK de .NET 10, una terminal PowerShell, Git y GitHub. Se consultó Microsoft Learn para verificar Dictionary<TKey, TValue>, HashSet<T>, las directivas using y Stopwatch. El proyecto se almacenó en la carpeta Practico-experimental03 del repositorio de Estructura de Datos. Identificación del problema Necesidad: registrar libros sin repetir el ISBN y conocer su estado. Usuario: persona responsable de la biblioteca. Entradas: ISBN, título, autor, categoría, año y lector. Procesos: registrar, indexar, buscar, prestar, devolver, eliminar, reportar y guardar. Salidas: libro encontrado, grupos por autor o categoría, estado disponible/prestado, lector responsable, resumen y mensajes de validación. Diseño orientado a objetos Libro es un record sellado e inmutable con propiedades tipadas Isbn, Titulo, Autor, Categoria y Anio; ToString genera una representación legible. Biblioteca es una clase sellada que mantiene las estructuras privadas y expone operaciones coherentes con el dominio. Program utiliza instrucciones de nivel superior para capturar las opciones y delegar las reglas en Biblioteca. Esta separación evita que el menú modifique directamente las colecciones y conserva la responsabilidad de cada componente. Función de las declaraciones using y los espacios de nombres La directiva using permite utilizar tipos de un espacio de nombres sin escribir su nombre completamente calificado (Microsoft, 2026). El archivo del proyecto activa ImplicitUsings, por lo que el SDK incorpora globalmente System, System.Collections.Generic, System.IO y System.Linq. System aporta Console, StringComparer, Action y Func; System.Collections.Generic aporta Dictionary, HashSet, List e interfaces de solo lectura; System.IO aporta File, Path e InvalidDataException; y System.Linq permite Select, OrderBy, ThenBy y FirstOrDefault. Además, using System.Diagnostics habilita Stopwatch para el benchmark; using System.Text.Json habilita JsonSerializer para la persistencia; y using PracticoExperimental03 permite que Program utilice Libro y Biblioteca. Estas declaraciones no crean estructuras: importan los tipos necesarios para emplearlas con nombres breves y legibles.

Estructuras de datos y función dentro de la solución		
Declaración	Contenido	Función específica
Dictionary<string, Libro> catalogo	ISBN -> Libro	Registro maestro y búsqueda exacta.
Dictionary<string, HashSet<string>> librosPorAutor	autor -> ISBN únicos	Mapa invertido para recuperar las obras de un autor.
Dictionary<string, HashSet<string>> librosPorCategoria	categoría -> ISBN únicos	Mapa invertido para consultas temáticas.
HashSet<string> disponibles	ISBN disponibles	Pertenencia rápida y ausencia de duplicados.
Dictionary<string, string> prestamos	ISBN -> lector	Responsable de cada préstamo activo.
Los mapas de autor y categoría combinan Dictionary con HashSet. El diccionario localiza el grupo y el conjunto impide que un ISBN aparezca repetido dentro de ese grupo. El catálogo conserva los objetos completos; los índices guardan solo identificadores para no duplicar todos los atributos. Procedimiento aplicado 1. Se creó el proyecto de consola Practico-experimental03 con .NET 10. 2. Se implementó Libro como record inmutable y se sobrescribió ToString. 3. Se implementó Biblioteca con catálogo, mapas invertidos, disponibilidad y préstamos. 4. RegistrarLibro valida los datos, rechaza ISBN duplicados y actualiza los tres índices. 5. BuscarPorIsbn consulta el catálogo; BuscarPorAutor y BuscarPorCategoria consultan los mapas invertidos. 6. PrestarLibro retira el ISBN del conjunto disponibles y lo agrega a prestamos; DevolverLibro realiza la operación inversa. 7. MostrarReporte ordena por categoría y título y presenta estado, lector y totales. 8. Guardar serializa libros y préstamos en JSON; Cargar reconstruye todos los índices. 9. Se incorporaron un modo de demostración, cinco pruebas automatizadas y un benchmark reproducible. 10. Se compiló con dotnet build y se ejecutaron los flujos de evidencia sin errores ni advertencias. Agente de inteligencia artificial utilizado Se utilizó Codex, basado en GPT-5, para interpretar la guía y la rúbrica, proponer la estructura inicial, revisar el código, orientar las pruebas, analizar la complejidad y apoyar la redacción del informe. El estudiante seleccionó el problema, organizó la carpeta en su repositorio, ejecutó el sistema, verificó los resultados y revisó el contenido. Se estima que aproximadamente el 85 % del código final recibió asistencia de IA; el porcentaje es una aproximación basada en el proceso de elaboración y se declara de forma transparente conforme a la guía.		
Resultados	La aplicación compiló correctamente con 0 errores y 0 advertencias. El catálogo inicial presentó cinco libros distribuidos en cuatro categorías. La consulta de Programación recuperó Effective Java y Fluent Python mediante el mapa	

librosPorCategoria, sin recorrer manualmente toda la tabla para formar el grupo.

El préstamo de Fluent Python a Ana López retiró su ISBN del conjunto disponibles y lo relacionó con la lectora en préstamos. Un segundo intento fue rechazado con el mensaje 'El libro no está disponible'. La devolución eliminó el préstamo y reincorporó el ISBN a disponibles. El reporte final mostró cinco libros disponibles y cero préstamos, lo que evidencia que las estructuras quedaron sincronizadas.

Las cinco pruebas automatizadas finalizaron correctamente: registro e índice por autor, rechazo de ISBN duplicado, préstamo y devolución, bloqueo de préstamo duplicado y persistencia JSON. El archivo guardado pudo cargarse reconstruyendo cinco libros y conservando el lector del préstamo.

Análisis del tiempo de ejecución

Operación	Estructura	Tiempo promedio	Justificación
Buscar por ISBN	Dictionary	$O(1)$	Acceso mediante clave hash.
Registrar	Dictionary + HashSet	$O(1)$ amortizado	Cantidad fija de inserciones; puede existir redimensionamiento.
Prestar/devolver	HashSet + Dictionary	$O(1)$ promedio	Pertenencia, eliminación e inserción por clave.
Buscar autor/categoría	Mapa invertido	$O(1 + k)$	Localiza el conjunto y devuelve k libros.
Reporte	Ordenamiento	$O(n \log n)$	Ordena todos los libros para presentar una salida legible.
Guardar/cargar	Recorrido	$O(n)$	Procesa todos los libros y préstamos.

El benchmark utilizó Stopwatch.GetTimestamp y la mediana de varias repeticiones. En esta ejecución, Dictionary tardó 0,034 μ s para 100 elementos y 0,033 μ s para 50 000, mientras el recorrido lineal aumentó de 2,300 μ s a 964,400 μ s. La cifra exacta depende del equipo, pero la tendencia evidencia que el acceso por clave permanece aproximadamente constante y el recorrido crece con n . Stopwatch se utiliza para medir tiempo transcurrido con precisión (Microsoft, s. f.-c).

Elementos	Dictionary (μ s)	Recorrido (μ s)	Relación
100	0,034	2,300	68,7x
1 000	0,022	26,000	1 198,2x
10 000	0,029	254,300	8 829,9x
50 000	0,033	964,400	29 048,2x

Análisis crítico de las estructuras en este problema

Ventajas. Dictionary permite localizar un libro por ISBN sin recorrer el catálogo; los mapas invertidos aceleran consultas repetidas por autor y categoría; HashSet evita ISBN duplicados y permite comprobar disponibilidad; y la combinación de estructuras representa con claridad las reglas del dominio. La clase Biblioteca mantiene las actualizaciones coordinadas y la persistencia JSON permite conservar los datos.

Desventajas. Los índices adicionales consumen más memoria que una lista única y cada registro o eliminación debe actualizar varias colecciones. Dictionary y

	HashSet no proporcionan por sí mismos orden alfabético, de modo que el reporte realiza un ordenamiento $O(n \log n)$. La búsqueda por autor o categoría es exacta después de normalizar espacios y mayúsculas, pero no corrige errores ortográficos ni realiza coincidencias parciales. Además, el ISBN identifica una edición y el modelo actual representa un solo ejemplar; una biblioteca con varias copias necesitaría un identificador de ejemplar. La elección queda justificada porque las operaciones centrales son consultas por claves, agrupaciones y pruebas de pertenencia. Una lista sería más simple y consumiría menos estructuras auxiliares, pero obligaría a recorrer hasta n libros para localizar un ISBN, formar una categoría o comprobar la disponibilidad. En este contexto, el beneficio de las consultas directas compensa la memoria y el cuidado adicional de mantener los índices.
Conclusiones	• Se cumplió el objetivo de implementar un sistema de biblioteca en C# mediante un record Libro y una clase Biblioteca que encapsula diccionarios, mapas invertidos y conjuntos. La compilación limpia, el flujo de préstamo y devolución y las cinco pruebas aprobadas demostraron que la solución funciona sin errores que impidan su uso. • Dictionary<string, Libro> resultó adecuado para el ISBN porque evita claves repetidas y permite acceso promedio $O(1)$. Los mapas por autor y categoría reducen consultas que, con una lista, serían $O(n)$, mientras HashSet<string> representa correctamente la disponibilidad sin duplicados. • El benchmark mostró que el recorrido lineal creció hasta 964,400 μs con 50 000 elementos, mientras la consulta por diccionario permaneció alrededor de centésimas de microsegundo. Esta diferencia respalda la elección de la estructura dentro del problema y no solo desde una descripción teórica. • La principal desventaja es mantener coherencia entre varias colecciones y utilizar memoria adicional. Como mejoras futuras se propone controlar varios ejemplares de un mismo ISBN, incorporar búsquedas parciales, registrar fechas de devolución y almacenar los datos en una base de datos.
Bibliografía	Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to algorithms (4th ed.). The MIT Press. https://mitpress.mit.edu/9780262046305/introduction-to-algorithms/ Microsoft. (s. f.-a). Dictionary<TKey, TValue> Clase (System.Collections.Generic). Microsoft Learn. Recuperado el 24 de agosto de 2026. https://learn.microsoft.com/es-es/dotnet/api/system.collections.generic.dictionary-2?view=net-10.0 Microsoft. (s. f.-b). HashSet<T> Clase (System.Collections.Generic). Microsoft Learn. Recuperado el 24 de agosto de 2026. https://learn.microsoft.com/es-es/dotnet/api/system.collections.generic.hashset-1?view=net-10.0 Microsoft. (2026, 26 de enero). La directiva using: Importar tipos de un espacio de nombres. Microsoft Learn. https://learn.microsoft.com/es-es/dotnet/csharp/language-reference/keywords/using-directive

Microsoft. (s. f.-c). Stopwatch Clase (System.Diagnostics). Microsoft Learn.
Recuperado el 24 de agosto de 2026. <https://learn.microsoft.com/es-es/dotnet/api/system.diagnostics.stopwatch?view=net-10.0>

Universidad Estatal Amazónica. (2026). Guía de Prácticas #03: Implementación de conjuntos y mapas [Guía académica].

Anexos

Evidencias del flujo real de ejecución

Figura 1.

Definición de las estructuras principales del sistema

```
Biblioteca.cs - estructuras principales

5 public sealed class Biblioteca
6 {
7     // Diccionario maestro: una clave ISBN identifica un solo libro.
8     private readonly Dictionary<string, Libro> catalogo = new(StringComparer.OrdinalIgnoreCase);
9
10    // Mapas invertidos: una clave de consulta se relaciona con varios ISBN únicos.
11    private readonly Dictionary<string, HashSet<string>> librosPorAutor =
12        new(StringComparer.OrdinalIgnoreCase);
13    private readonly Dictionary<string, HashSet<string>> librosPorCategoria =
14        new(StringComparer.OrdinalIgnoreCase);
15
16    // Conjunto para comprobar rápidamente si un ISBN está disponible.
17    private readonly HashSet<string> disponibles = new(StringComparer.OrdinalIgnoreCase);
18
19    // Diccionario que relaciona cada ISBN prestado con el lector responsable.
20    private readonly Dictionary<string, string> prestamos = new(StringComparer.OrdinalIgnoreCase);
21
22    public IReadOnlyDictionary<string, Libro> Catalogo => catalogo;
23    public IReadOnlySet<string> Disponibles => disponibles;
24    public IReadOnlyDictionary<string, string> Prestamos => prestamos;
25
26    private static string Normalizar(string texto) =>
27        string.Join(' ', texto.Trim().ToLowerInvariant()
28            .Split(' ', StringSplitOptions.RemoveEmptyEntries));
29
30    private static void AgregarAIndice(
31        Dictionary<string, HashSet<string>> indice,
```

Nota. La captura muestra Dictionary y HashSet declarados como campos privados de Biblioteca. Elaboración propia en C# (2026).

Figura 2.

Catálogo inicial y reportería del sistema

```
Terminal - dotnet run -- --demo

DEMOSTRACIÓN REPRODUCIBLE - PRÁCTICA 03

===== CATÁLOGO DE LA BIBLIOTECA =====
ISBN          TÍTULO                      CATEGORÍA      ESTADO      LECTOR
-----
9780262046305 Introduction to Algorithms      Algoritmos     Disponible  -
9788417347444 El infinito en un junco        Ensayo         Disponible  -
9780134757599 Refactoring                    Ingeniería de s... Disponible  -
9780134685991 Effective Java                Programación   Disponible  -
9781492056355 Fluent Python                  Programación   Disponible  -
-----
Total: 5 | Disponibles: 5 | Prestados: 0 | Autores: 5 | Categorías: 4
```

Nota. Salida real del modo demostración. Se visualizan ISBN, título, categoría, estado y totales. Elaboración propia (2026).

Figura 3.

Consulta por categoría mediante un mapa invertido

```
Terminal - consulta y préstamo

Préstamo: Préstamo registrado correctamente.
Consulta por categoría Programación:
9780134685991 | Effective Java | Joshua Bloch | Programación | 2018
9781492056355 | Fluent Python | Luciano Ramalho | Programación | 2022
Validación de préstamo duplicado: El libro no está disponible.
```

Nota. El mapa Programación recupera dos libros y el sistema registra el préstamo de Fluent Python. Elaboración propia (2026).

Figura 4.
Validación del préstamo y reporte actualizado

```

Terminal - préstamo y validación

Seleccione una opción:
===== CATÁLOGO DE LA BIBLIOTECA =====
ISBN          TÍTULO          CATEGORÍA      ESTADO      LECTOR
-----
9780262046305  Introduction to Algorithms  Algoritmos     Disponible  -
9788417347444  El infinito en un junco     Ensayo         Disponible  -
9780134757599  Refactoring                 Ingeniería de s... Disponible  -
9780134685991  Effective Java              Programación    Disponible  -
9781492056355  Fluent Python               Programación    Prestado    Carlos Jamil Papa
-----
Total: 5 | Disponibles: 4 | Prestados: 1 | Autores: 5 | Categorías: 4

===== SISTEMA DE BIBLIOTECA =====
1. Registrar libro
2. Buscar por ISBN
3. Buscar por autor
4. Buscar por categoría
5. Prestar libro
6. Devolver libro
7. Mostrar reporte completo
8. Eliminar libro
9. Guardar datos en JSON
0. Salir
Seleccione una opción: ISBN: Nombre del lector: El libro no está disponible.

```

Nota. La evidencia muestra el flujo interactivo y las validaciones aplicadas a disponibilidad y préstamos. Elaboración propia (2026).

Figura 5.
Resultado de las pruebas automatizadas

```

Terminal - dotnet run -- --pruebas

[OK] Registro e índice por autor
[OK] Rechazo de ISBN duplicado
[OK] Préstamo y devolución
[OK] Bloqueo de préstamo duplicado
[OK] Persistencia JSON

Resultado: 5/5 pruebas aprobadas.

```

Nota. Las cinco pruebas finalizaron correctamente, incluida la persistencia JSON. Elaboración propia (2026).

Figura 6.
Medición del tiempo de búsqueda

```

Terminal - dotnet run -- --benchmark

BENCHMARK DE BÚSQUEDA POR ISBN
Elementos | Dictionary (µs) | Recorrido (µs) | Relación
-----
100 | 0,034 | 2,300 | 68,7x
1.000 | 0,022 | 26,000 | 1198,2x
10.000 | 0,029 | 254,300 | 8829,9x
50.000 | 0,033 | 964,400 | 29048,2x

```

Nota. Benchmark real que compara Dictionary con un recorrido lineal para cuatro tamaños de catálogo. Elaboración propia (2026).

Código fuente completo

La versión ejecutada se incluye a continuación y coincide con la carpeta Practico-experimental03 del repositorio.

Anexo 7. Libro.cs

```

namespace PracticoExperimental03;

public sealed record Libro(
    string Isbn,
    string Titulo,
    string Autor,
    string Categoria,
    int Anio)

```

```

{
    public override string ToString() =>
        $"{Isbn} | {Titulo} | {Autor} | {Categoria} | {Anio}";
}

Anexo 8. Biblioteca.cs
using System.Text.Json;

namespace PracticoExperimental03;

public sealed class Biblioteca
{
    // Diccionario maestro: una clave ISBN identifica un solo libro.
    private readonly Dictionary<string, Libro> catalogo =
        new(StringComparer.OrdinalIgnoreCase);

    // Mapas invertidos: una clave de consulta se relaciona con varios ISBN únicos.
    private readonly Dictionary<string, HashSet<string>> librosPorAutor =
        new(StringComparer.OrdinalIgnoreCase);
    private readonly Dictionary<string, HashSet<string>> librosPorCategoria =
        new(StringComparer.OrdinalIgnoreCase);

    // Conjunto para comprobar rápidamente si un ISBN está disponible.
    private readonly HashSet<string> disponibles = new(StringComparer.OrdinalIgnoreCase);

    // Diccionario que relaciona cada ISBN prestado con el lector responsable.
    private readonly Dictionary<string, string> prestamos =
        new(StringComparer.OrdinalIgnoreCase);

    public IReadOnlyDictionary<string, Libro> Catalogo => catalogo;
    public IReadOnlySet<string> Disponibles => disponibles;
    public IReadOnlyDictionary<string, string> Prestamos => prestamos;

    private static string Normalizar(string texto) =>
        string.Join(' ', texto.Trim().ToLowerInvariant()
            .Split(' ', StringSplitOptions.RemoveEmptyEntries));

    private static void AgregarAIndice(
        Dictionary<string, HashSet<string>> indice,
        string clave,
        string isbn)
    {
        if (!indice.TryGetValue(clave, out HashSet<string>? codigos))
        {
            codigos = new HashSet<string>(StringComparer.OrdinalIgnoreCase);
            indice[clave] = codigos;
        }

        codigos.Add(isbn);
    }

    private static void QuitarDeIndice(
        Dictionary<string, HashSet<string>> indice,
        string clave,
        string isbn)
    {
        if (!indice.TryGetValue(clave, out HashSet<string>? codigos))
            return;

        codigos.Remove(isbn);
        if (codigos.Count == 0)
            indice.Remove(clave);
    }

    public bool RegistrarLibro(Libro libro, out string mensaje)
    {
        if (string.IsNullOrEmpty(libro.Isbn) ||
            string.IsNullOrEmpty(libro.Titulo) ||
            string.IsNullOrEmpty(libro.Autor) ||
            string.IsNullOrEmpty(libro.Categoria) ||
            libro.Anio <= 0)
        {
            mensaje = "Los datos del libro son incompletos o inválidos.";
            return false;
        }

        if (!catalogo.TryAdd(libro.Isbn.Trim(), libro))
        {
            mensaje = $"El ISBN {libro.Isbn} ya se encuentra registrado.";
            return false;
        }

        disponibles.Add(libro.Isbn);
        AgregarAIndice(librosPorAutor, Normalizar(libro.Autor), libro.Isbn);
        AgregarAIndice(librosPorCategoria, Normalizar(libro.Categoria), libro.Isbn);
        mensaje = "Libro registrado correctamente.";
    }
}

```



```

    return true;
}

public bool EliminarLibro(string isbn, out string mensaje)
{
    if (prestamos.ContainsKey(isbn))
    {
        mensaje = "No se puede eliminar un libro que está prestado.";
        return false;
    }

    if (!catalogo.Remove(isbn, out Libro? libro))
    {
        mensaje = "No existe un libro con ese ISBN.";
        return false;
    }

    disponibles.Remove(isbn);
    QuitarDeIndice(librosPorAutor, Normalizar(libro.Autor), isbn);
    QuitarDeIndice(librosPorCategoria, Normalizar(libro.Categoria), isbn);
    mensaje = "Libro eliminado correctamente.";
    return true;
}

public Libro? BuscarPorIsbn(string isbn) =>
    catalogo.GetValueOrDefault(isbn);

public IReadOnlyList<Libro> BuscarPorAutor(string autor) =>
    BuscarEnIndice(librosPorAutor, Normalizar(autor));

public IReadOnlyList<Libro> BuscarPorCategoria(string categoria) =>
    BuscarEnIndice(librosPorCategoria, Normalizar(categoria));

private IReadOnlyList<Libro> BuscarEnIndice(
    Dictionary<string, HashSet<string>> indice,
    string clave)
{
    if (!indice.TryGetValue(clave, out HashSet<string>? codigos))
        return [];

    return codigos.Select(isbn => catalogo[isbn])
        .OrderBy(libro => libro.Titulo)
        .ToList();
}

public bool PrestarLibro(string isbn, string lector, out string mensaje)
{
    if (!catalogo.ContainsKey(isbn))
    {
        mensaje = "No existe un libro con ese ISBN.";
        return false;
    }

    if (string.IsNullOrEmpty(lector))
    {
        mensaje = "El nombre del lector es obligatorio.";
        return false;
    }

    if (!disponibles.Remove(isbn))
    {
        mensaje = "El libro no está disponible.";
        return false;
    }

    prestamos[isbn] = lector.Trim();
    mensaje = "Préstamo registrado correctamente.";
    return true;
}

public bool DevolverLibro(string isbn, out string mensaje)
{
    if (!prestamos.Remove(isbn))
    {
        mensaje = "El libro no registra un préstamo activo.";
        return false;
    }

    disponibles.Add(isbn);
    mensaje = "Devolución registrada correctamente.";
    return true;
}

public void MostrarReporte()
{

```

```

        Console.WriteLine("\n===== CATÁLOGO DE LA BIBLIOTECA
=====");
        Console.WriteLine($"{"ISBN",-15} {"TÍTULO",-28} {"CATEGORÍA",-18} {"ESTADO",-12}
LECTOR"}");
        Console.WriteLine(new string('-', 95));

        foreach (Libro libro in catalogo.Values
            .OrderBy(libro => libro.Categoria)
            .ThenBy(libro => libro.Titulo))
        {
            string estado = disponibles.Contains(libro.Isbn) ? "Disponible" : "Prestado";
            string lector = prestamos.GetValueOrDefault(libro.Isbn, "-");
            Console.WriteLine($"{"Recortar(libro.Isbn, 15),-15} " +
                $"{"Recortar(libro.Titulo, 28),-28} " +
                $"{"Recortar(libro.Categoria, 18),-18} " +
                $"{"estado,-12} {lector}");
        }

        Console.WriteLine(new string('-', 95));
        Console.WriteLine($"Total: {catalogo.Count} | Disponibles: {disponibles.Count} | "
+
            $"Prestados: {prestamos.Count} | Autores: {librosPorAutor.Count}
| " +
            $"Categorías: {librosPorCategoria.Count}");
    }

    private static string Recortar(string texto, int longitud) =>
        texto.Length <= longitud ? texto : texto[..(longitud - 3)] + "...";

    public void Guardar(string ruta)
    {
        DatosBiblioteca datos = new(catalogo.Values.ToList(), new Dictionary<string,
string>(prestamos));
        string json = JsonSerializer.Serialize(datos, new JsonSerializerOptions {
WriteIndented = true });
        File.WriteAllText(ruta, json);
    }

    public static Biblioteca Cargar(string ruta)
    {
        string json = File.ReadAllText(ruta);
        DatosBiblioteca datos = JsonSerializer.Deserialize<DatosBiblioteca>(json)
            ?? throw new InvalidDataException("El archivo no contiene datos válidos.");

        Biblioteca biblioteca = new();
        foreach (Libro libro in datos.Libros)
            biblioteca.RegistrarLibro(libro, out _);
        foreach ((string isbn, string lector) in datos.Prestamos)
            biblioteca.PrestarLibro(isbn, lector, out _);
        return biblioteca;
    }

    private sealed record DatosBiblioteca(
        List<Libro> Libros,
        Dictionary<string, string> Prestamos);
}

```

Anexo 9. Program.cs

```

using System.Diagnostics;
using PracticoExperimental03;

Console.OutputEncoding = System.Text.Encoding.UTF8;

if (args.Contains("--demo"))
{
    EjecutarDemostracion();
    return;
}

if (args.Contains("--pruebas"))
{
    EjecutarPruebas();
    return;
}

if (args.Contains("--benchmark"))
{
    EjecutarBenchmark();
    return;
}

Biblioteca biblioteca = CrearBibliotecaInicial();
int opcion;

do
{

```

```

Console.WriteLine("\n===== SISTEMA DE BIBLIOTECA =====");
Console.WriteLine("1. Registrar libro");
Console.WriteLine("2. Buscar por ISBN");
Console.WriteLine("3. Buscar por autor");
Console.WriteLine("4. Buscar por categoría");
Console.WriteLine("5. Prestar libro");
Console.WriteLine("6. Devolver libro");
Console.WriteLine("7. Mostrar reporte completo");
Console.WriteLine("8. Eliminar libro");
Console.WriteLine("9. Guardar datos en JSON");
Console.WriteLine("0. Salir");
Console.Write("Seleccione una opción: ");

if (!int.TryParse(Console.ReadLine(), out opcion))
{
    Console.WriteLine("Debe ingresar un número.");
    continue;
}

switch (opcion)
{
    case 1:
        RegistrarDesdeConsola(biblioteca);
        break;
    case 2:
        Console.Write("ISBN: ");
        Libro? libro = biblioteca.BuscarPorIsbn(Console.ReadLine() ?? "");
        Console.WriteLine(libro is null ? "No se encontró el libro." : libro);
        break;
    case 3:
        Console.Write("Autor: ");
        MostrarResultados(biblioteca.BuscarPorAutor(Console.ReadLine() ?? ""));
        break;
    case 4:
        Console.Write("Categoría: ");
        MostrarResultados(biblioteca.BuscarPorCategoria(Console.ReadLine() ?? ""));
        break;
    case 5:
        Console.Write("ISBN: ");
        string isbnPrestamo = Console.ReadLine() ?? "";
        Console.Write("Nombre del lector: ");
        biblioteca.PrestarLibro(isbnPrestamo, Console.ReadLine() ?? "", out string
mensajePrestamo);
        Console.WriteLine(mensajePrestamo);
        break;
    case 6:
        Console.Write("ISBN: ");
        biblioteca.DevolverLibro(Console.ReadLine() ?? "", out string
mensajeDevolucion);
        Console.WriteLine(mensajeDevolucion);
        break;
    case 7:
        biblioteca.MostrarReporte();
        break;
    case 8:
        Console.Write("ISBN: ");
        biblioteca.EliminarLibro(Console.ReadLine() ?? "", out string
mensajeEliminar);
        Console.WriteLine(mensajeEliminar);
        break;
    case 9:
        biblioteca.Guardar("biblioteca.json");
        Console.WriteLine("Datos guardados en biblioteca.json.");
        break;
    case 0:
        Console.WriteLine("Programa finalizado.");
        break;
    default:
        Console.WriteLine("Opción no válida.");
        break;
}
} while (opcion != 0);

static Biblioteca CrearBibliotecaInicial()
{
    Biblioteca biblioteca = new();
    Libro[] libros =
    [
        new("9780134685991", "Effective Java", "Joshua Bloch", "Programación", 2018),
        new("9781492056355", "Fluent Python", "Luciano Ramalho", "Programación", 2022),
        new("9780262046305", "Introduction to Algorithms", "Thomas H. Cormen",
"Algoritmos", 2022),
        new("9780134757599", "Refactoring", "Martin Fowler", "Ingeniería de software",
2018),
        new("9788417347444", "El infinito en un junco", "Irene Vallejo", "Ensayo", 2019)
    ];

    foreach (Libro libro in libros)
        biblioteca.RegistrarLibro(libro, out _);
    return biblioteca;
}

```

```

}

static void RegistrarDesdeConsola(Biblioteca biblioteca)
{
    Console.WriteLine("ISBN: ");
    string isbn = Console.ReadLine() ?? "";
    Console.WriteLine("Título: ");
    string titulo = Console.ReadLine() ?? "";
    Console.WriteLine("Autor: ");
    string autor = Console.ReadLine() ?? "";
    Console.WriteLine("Categoría: ");
    string categoria = Console.ReadLine() ?? "";
    Console.WriteLine("Año: ");
    int.TryParse(Console.ReadLine(), out int anio);
    biblioteca.RegistrarLibro(new Libro(isbn, titulo, autor, categoria, anio), out string
mensaje);
    Console.WriteLine(mensaje);
}

static void MostrarResultados(IReadOnlyList<Libro> libros)
{
    if (libros.Count == 0)
    {
        Console.WriteLine("No se encontraron coincidencias.");
        return;
    }

    foreach (Libro libro in libros)
        Console.WriteLine(libro);
}

static void EjecutarDemostracion()
{
    Biblioteca biblioteca = CrearBibliotecaInicial();
    Console.WriteLine("DEMOSTRACIÓN REPRODUCIBLE - PRÁCTICA 03");
    biblioteca.MostrarReporte();

    biblioteca.PrestarLibro("9781492056355", "Ana López", out string prestamo);
    Console.WriteLine($"Prestamo: {prestamo}");
    Console.WriteLine("Consulta por categoría Programación:");
    MostrarResultados(biblioteca.BuscarPorCategoria("programación"));

    biblioteca.PrestarLibro("9781492056355", "Carlos Ruiz", out string duplicado);
    Console.WriteLine($"Validación de préstamo duplicado: {duplicado}");

    biblioteca.DevolverLibro("9781492056355", out string devolucion);
    Console.WriteLine($"Devolución: {devolucion}");
    biblioteca.MostrarReporte();
}

static void EjecutarPruebas()
{
    int aprobadas = 0;
    Probar("Registro e índice por autor", () =>
    {
        Biblioteca b = CrearBibliotecaInicial();
        return b.BuscarPorAutor(" JOSHUA BLOCH ").Count == 1;
    }, ref aprobadas);
    Probar("Rechazo de ISBN duplicado", () =>
    {
        Biblioteca b = CrearBibliotecaInicial();
        return !b.RegistrarLibro(new Libro("9780134685991", "Otro", "Autor", "Tema",
2024), out _);
    }, ref aprobadas);
    Probar("Préstamo y devolución", () =>
    {
        Biblioteca b = CrearBibliotecaInicial();
        bool prestar = b.PrestarLibro("9781492056355", "Ana", out _);
        bool devolver = b.DevolverLibro("9781492056355", out _);
        return prestar && devolver && b.Disponibles.Contains("9781492056355");
    }, ref aprobadas);
    Probar("Bloqueo de préstamo duplicado", () =>
    {
        Biblioteca b = CrearBibliotecaInicial();
        b.PrestarLibro("9781492056355", "Ana", out _);
        return !b.PrestarLibro("9781492056355", "Luis", out _);
    }, ref aprobadas);
    Probar("Persistencia JSON", () =>
    {
        string ruta = Path.Combine(Path.GetTempPath(), $"biblioteca-
{Guid.NewGuid()}.json");
        try
        {
            Biblioteca b = CrearBibliotecaInicial();
            b.PrestarLibro("9781492056355", "Ana", out _);
            b.Guardar(ruta);
            Biblioteca copia = Biblioteca.Cargar(ruta);
            return copia.Catalogo.Count == 5 && copia.Prestamos["9781492056355"] == "Ana";
        }
    });
}

```



```

    }
    finally
    {
        if (File.Exists(ruta)) File.Delete(ruta);
    }
}, ref aprobadas);

Console.WriteLine($"\\nResultado: {aprobadas}/5 pruebas aprobadas.");
}

static void Probar(string nombre, Func<bool> prueba, ref int aprobadas)
{
    bool resultado = prueba();
    Console.WriteLine($"[{(resultado ? "OK" : "ERROR")}] {nombre}");
    if (resultado) aprobadas++;
}

static void EjecutarBenchmark()
{
    Console.WriteLine("BENCHMARK DE BÚSQUEDA POR ISBN");
    Console.WriteLine($"{"Elementos",10} | {"Dictionary (μs)",16} | {"Recorrido (μs)",16} | {"Relación"}");
    Console.WriteLine(new string('-', 66));

    foreach (int cantidad in new[] { 100, 1_000, 10_000, 50_000 })
    {
        Dictionary<string, Libro> datos = new();
        for (int i = 0; i < cantidad; i++)
            datos[$"ISBN-{i:000000}"] = new Libro($"ISBN-{i:000000}", $"Libro {i}", "Autor", "Categoria", 2024);

        string objetivo = $"ISBN-{cantidad - 1:000000}";
        double directo = Medir(() =>
        {
            Libro? encontrado = null;
            for (int i = 0; i < 1_000; i++)
                encontrado = datos.GetValueOrDefault(objetivo);
            GC.KeepAlive(encontrado);
        }, 300, 1_000);
        double lineal = Medir(
            () => datos.Values.FirstOrDefault(libro => libro.Isbn == objetivo),
            300, 1);
        Console.WriteLine($"{"cantidad,10:N0} | {"directo,16:F3} | {"lineal,16:F3} | {"lineal / directo,7:F1}x");
    }
}

static double Medir(Action accion, int repeticiones, int operacionesPorMuestra)
{
    long[] muestras = new long[repeticiones];
    for (int i = 0; i < repeticiones; i++)
    {
        long inicio = Stopwatch.GetTimestamp();
        accion();
        muestras[i] = Stopwatch.GetTimestamp() - inicio;
    }

    Array.Sort(muestras);
    return muestras[muestras.Length / 2] * 1_000_000.0 /
        Stopwatch.Frequency /
        operacionesPorMuestra;
}

```

Repositorio: <https://github.com/CarlosPapa18/ESTRUCTURA-DE-DATOS--CARLOS/tree/main/Practico-experimental03>



Firma del estudiante